



- ▶ Final Projects Due Mon, 4/27 by midnight
- ▶ Term Project Peer Evaluations - Due Mon, 4/27 by midnight
 - ▶ These are **confidential** and will not be shared with anyone
 - ▶ These should take less than 10 minutes to fill out and will be used to make any adjustments in grading
 - ▶ Not responding will impact your term project grade
 - ▶ These will be made available next week

Architecting for the Cloud

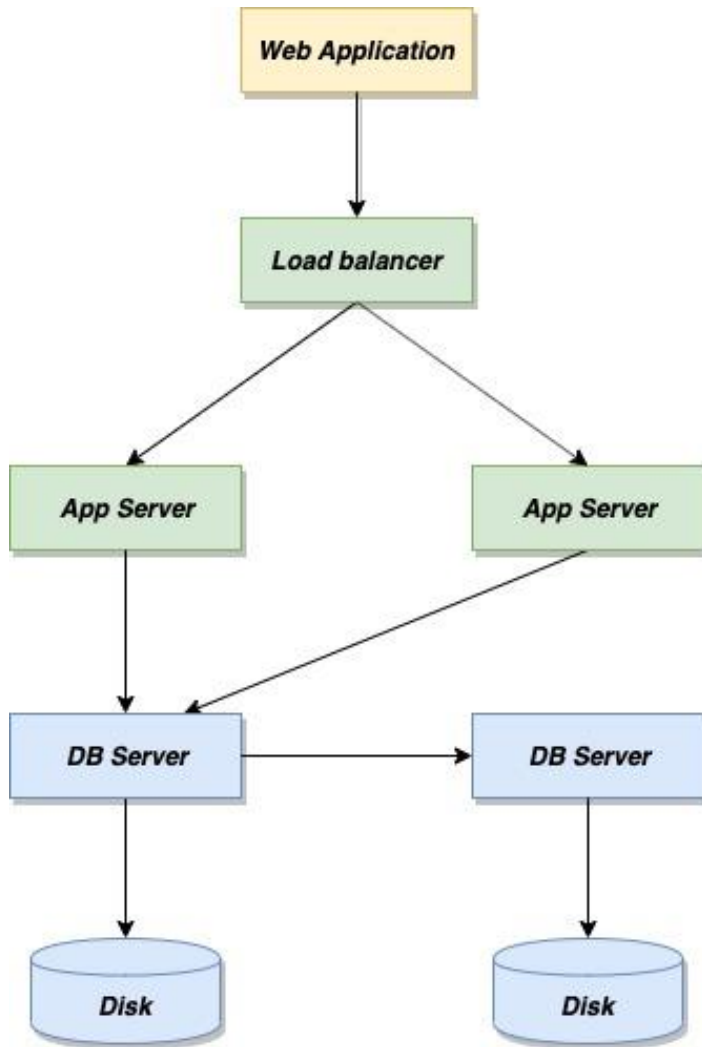
Engineering Cloud Software Systems

Department of Software Engineering
Rochester Institute of Technology



A Design Review Exercise

3



- ▶ You are designing a new cloud-based application for an organization adopting AWS
- ▶ The development team understands basic cloud services but has limited production experience.
- ▶ An initial architecture has been proposed and is functional, but not yet production-ready.
- ▶ Your task is to review the architecture and recommend improvements before it goes live
- ▶ If this system were deployed today, what are your recommended best practices as an architect?

Cloud Architecture Best Practices

1. Plan for scale and service limits early
2. Select regions based on cost, latency, and service availability
3. Design with cost awareness from the beginning
4. Design For Failure, Not Perfection
5. Use Auto-scaling to handle demand and recover from failures
6. Automate operational and administrative tasks
7. Manage infrastructure using Infrastructure as Code (IaC)
8. Apply strong identity and access management
9. Establish CI/CD pipelines for consistent deployments
10. Monitor and observe application and system behavior
11. Prefer managed services where appropriate
12. Regularly evaluate and evolve the architecture as usage grows



Architecting
for the Cloud

Best Practice #1: Plan for scale and service limits early

- ▶ Cloud services have limits (API calls, throughput, connections)
- ▶ Scaling isn't automatic unless designed for
- ▶ Why it matters: Apps often fail during success; hitting limits causes outages
- ▶ Avoid surprises:
 - ▶ Run load tests to know capacity and breaking points
 - ▶ Plan upgrades early based on growth goals
- ▶ Examples:
 - ▶ EC2 instance limits reached during rapid auto-scaling
 - ▶ S3 request rate limits causing latency under heavy uploads
 - ▶ RDS connection limits exceeded during peak traffic



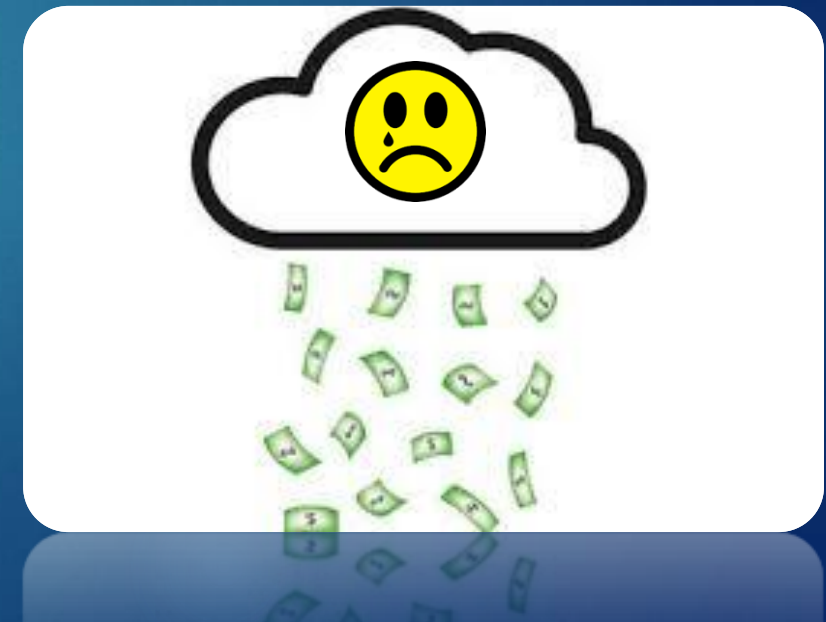
Best Practice #2: Select regions based on cost, latency, and service availability

- ▶ A very common mistake is to choose an AWS region simply based on the location of your customers
- ▶ Not all AWS regions are created equal, there are substantial differences in price and feature availability
- ▶ If you think you'll use a certain AWS service in the near future, make sure it's available in your chosen region
- ▶ Some configurations can cost almost double, depending on the region you choose
 - ▶ Data transfer in Sao Paulo is 177% more expensive compared to N. Virginia
- ▶ Even within the US, some EC2 instances (like a t2.large) in N. California will cost you 30% more compared to N. Virginia



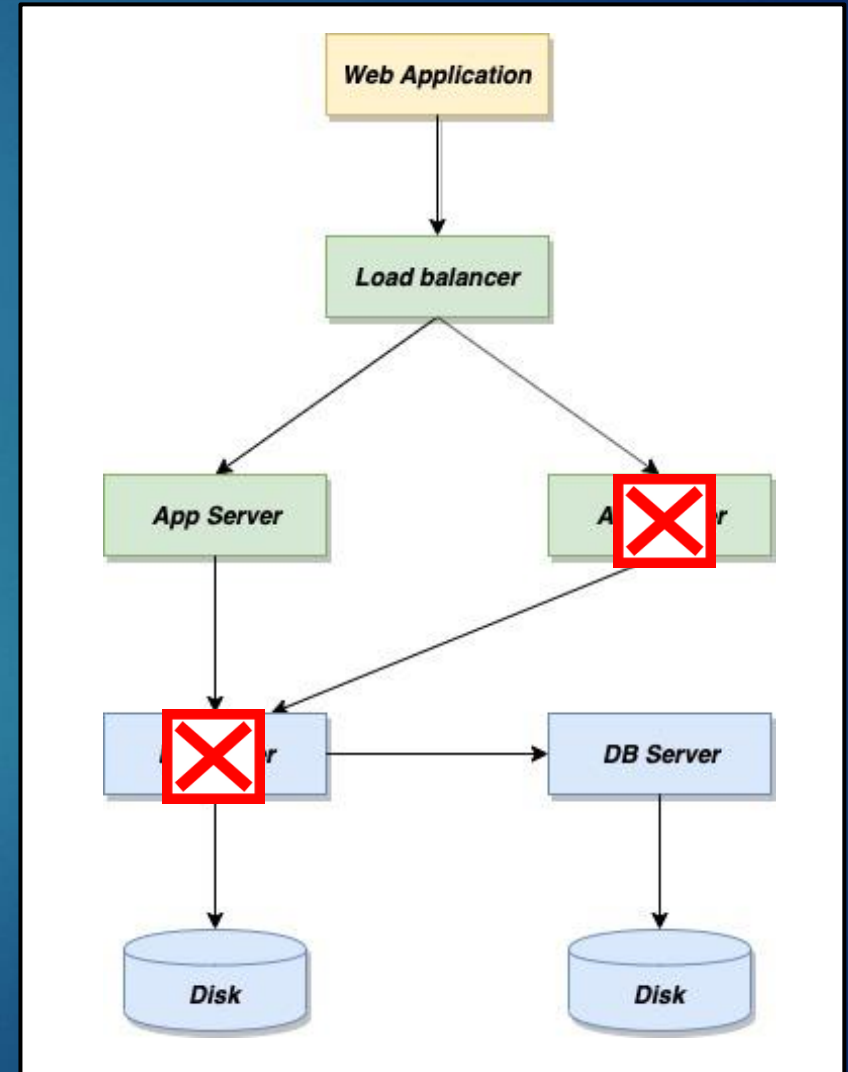
Best Practice #3: Design with cost awareness from the beginning

- ▶ An application that costs more than it earns will erode profits
Spending \$2,000/month on resources for an app that makes \$1,000/month is unsustainable
- ▶ Choose the right instances:
 - ▶ How many do you need?
 - ▶ What size is appropriate?
- ▶ Track and calculate costs - AWS pricing can be complex, but understanding resource usage is key
- ▶ Account for all cost factors, such as:
 - ▶ Data transfer (ingress/egress)
 - ▶ ELB processing
 - ▶ Instance hours
 - ▶ EBS/S3 storage
 - ▶ API calls
 - ▶ Lambda executions and memory



Best Practice #4: Design For Failure, Not Perfection

- ▶ Imagine your web application goes down overnight because the virtual machine fails
- ▶ For 8 hours, customers can't access your site
- ▶ Frustrated users look for alternatives and stop buying from you
- ▶ The result? Lost revenue and damaged reputation is a disaster for any business



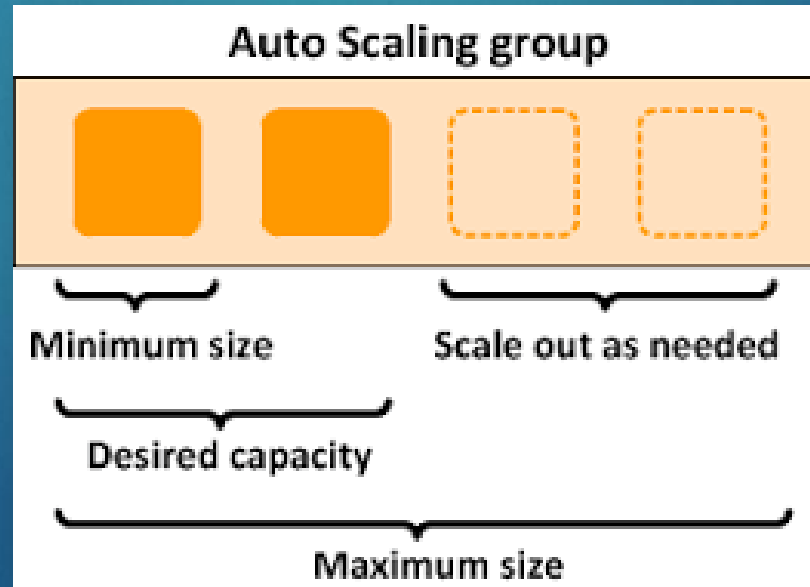
Best Practice #4: Design For Failure, Not Perfection

- ▶ Outages can occur due to:
 - ▶ Host software issues
 - ▶ The host OS or virtualization layer crashes, causing the VM to fail
 - ▶ Hardware failures
 - ▶ Computing, storage, or networking components on the physical host break down
 - ▶ Data center issues
 - ▶ Loss of network connectivity, power supply, or cooling systems
- ▶ If the physical host's hardware fails, all EC2 instances on that host will go down



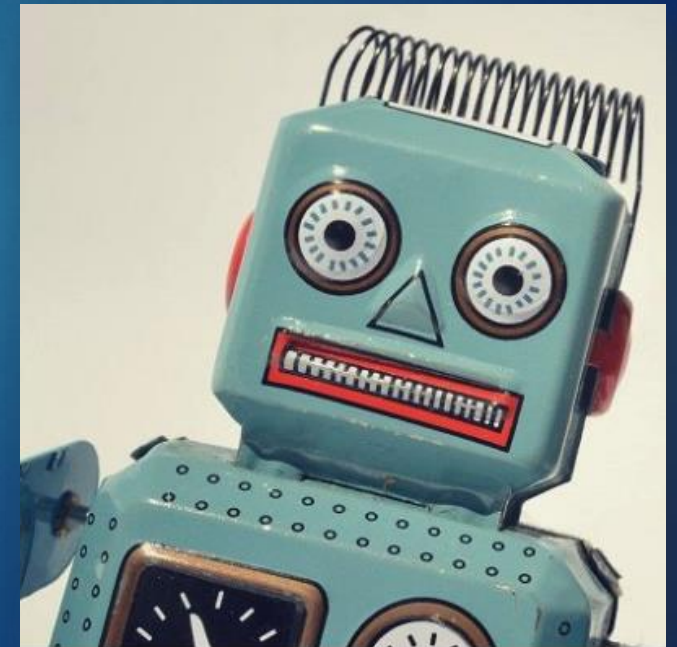
Best Practice #5: Use Auto-scaling to handle demand and recover from failures

- ▶ Launch every EC2 instance in an Auto Scaling Group, even single instances
- ▶ Auto-Scaling Groups monitor instances and replace failed ones automatically
- ▶ Scale based on alarms for metrics like CPU usage or load balancer requests
- ▶ Define actions when thresholds are met, such as adding more instances
- ▶ Common use case: fixed-size groups for resiliency (e.g., min = 1, max = 1)



Best Practice #6: Automate Operational and Administrative Tasks

- ▶ Reduce manual steps in setup, deployment, and recovery
- ▶ Automate anything repetitive or error-prone
- ▶ Why it matters?
 - ▶ Manual processes don't scale
 - ▶ Human error is a leading cause of outages and downtime
- ▶ Examples:
 - ▶ Automated environment setup scripts
 - ▶ Scheduled backups and cleanup tasks



Best Practice #7: Manage Infrastructure using Infrastructure as Code (IaC)

- ▶ Define infrastructure through code, not manual console steps
- ▶ Manage infrastructure changes like software updates for consistency
- ▶ Ensures repeatability, version control, and easy rollback
- ▶ Keeps environments consistent and auditable
- ▶ Use tools like Terraform or AWS CloudFormation for provisioning
- ▶ Quickly recreate environments for testing or disaster recovery



Best Practice #8: Apply Strong Identity and Access Management

- ▶ Grant only the minimum permissions needed for each task
- ▶ Regularly review and update access policies to maintain security
- ▶ Avoid overly broad permissions as they pose a serious security risk
- ▶ IAM misconfigurations can expose critical resources
- ▶ Use separate roles for developers and administrators
- ▶ Prefer IAM roles over long-lived credentials for better security



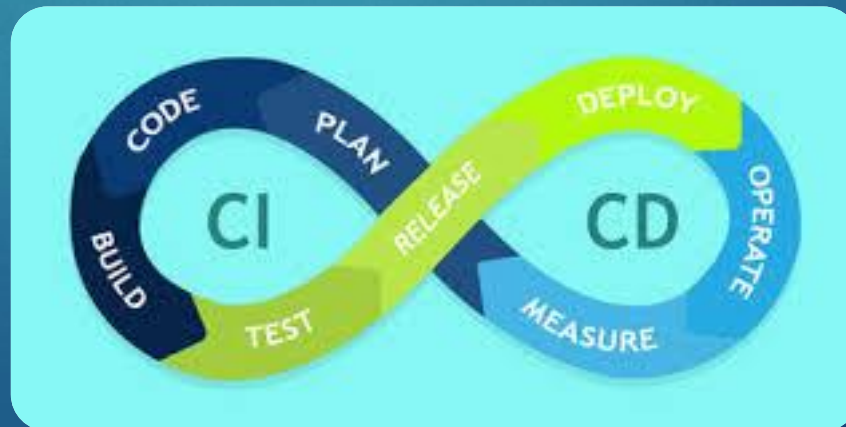
Best Practice #8: Apply Strong Identity and Access Management

- ▶ One of the most frequent and serious errors: making S3 buckets publicly accessible
- ▶ S3 buckets are secure by default, but access is often loosened during development
- ▶ Permissions often remain unchanged when going live, leading to high-profile breaches
- ▶ Always follow the principle of least privilege:
 - ▶ Grant access only to those who are trained and authorized
 - ▶ Review and tighten permissions before production



Best Practice #9: Establish CI/CD Pipelines for Consistent Deployments

- ▶ Automate build, test, and deployment processes to reduce manual steps
- ▶ Speeds up deployments and improves reliability
- ▶ Minimizes configuration drift and human error
- ▶ Use tools like GitHub Actions for deploying infrastructure and application code
- ▶ Include automated testing before production releases

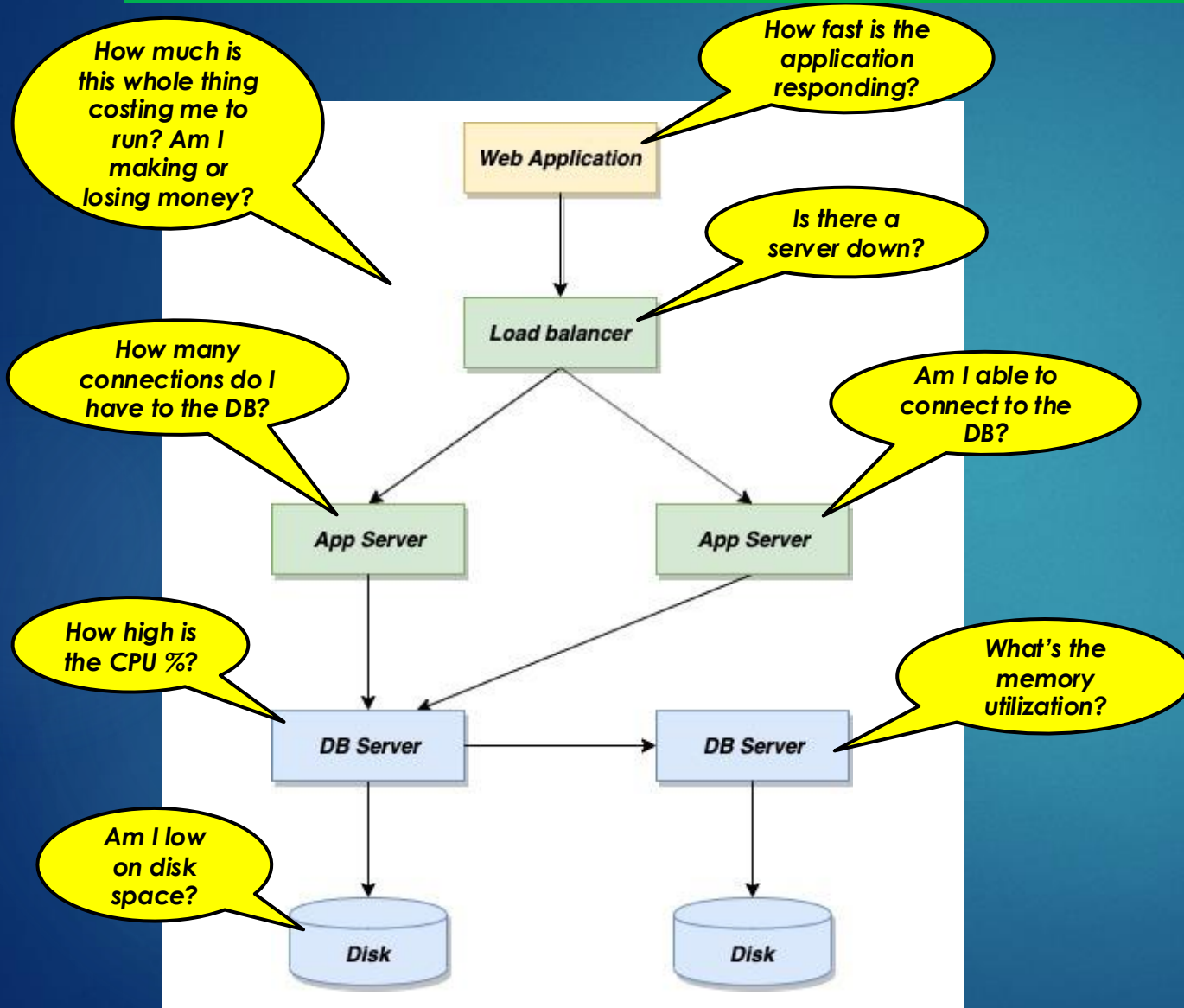


Best Practice #10: Monitor and Observe Application and System Behavior

- ▶ Track metrics, logs, and alerts that directly impact user experience, not just infrastructure health
- ▶ Include application-level metrics such as error rates, latency, and transaction success rates
- ▶ Implement distributed tracing to understand request flow across services
- ▶ Use centralized logging for visibility into failures and performance bottlenecks
- ▶ You can't manage what you don't measure as monitoring enables early detection and faster recovery
- ▶ Examples:
 - ▶ CloudWatch alarms for error rate or response time thresholds
 - ▶ Custom dashboards for critical workflows
 - ▶ Log insights to identify anomalies and trends



Best Practice #10: Monitor and Observe Application and System Behavior



▶ Customer Experience Metrics

- ▶ Examples: response times, page load times, error 500s, etc.

▶ System Metrics

- ▶ Examples: CPU Utilization, memory utilization, disk I/O, queue length, etc.

▶ Business Metrics

- ▶ Examples: Revenue from application (e.g. orders submitted) vs. how much this all costs to run in the cloud

▶ Observability Tip

- ▶ Use tools like Amazon CloudWatch and AWS X-Ray to monitor these metrics, trace requests, and set alerts for critical thresholds

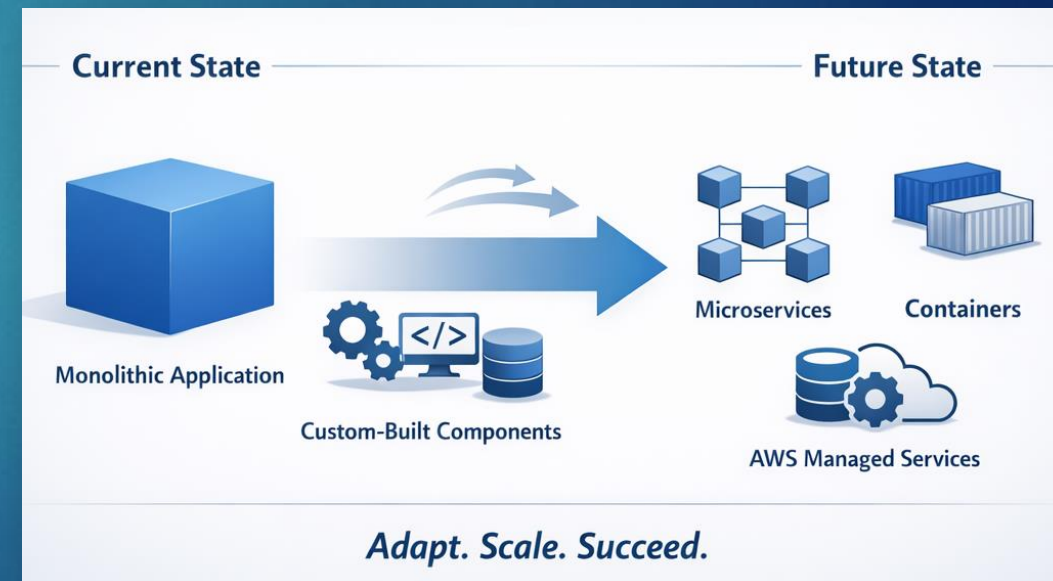
Best Practice #11: Prefer Managed Services Where Appropriate

- ▶ Use AWS-managed services instead of self-managed solutions whenever possible.
- ▶ Focus on business logic, not infrastructure maintenance
- ▶ Managed services reduce operational overhead
 - ▶ AWS handles scaling, patching, and availability
- ▶ Examples:
 - ▶ Amazon RDS for databases instead of self-managed DB servers
 - ▶ AWS Lambda for compute instead of always-on EC2 instances



Best Practice #12: Regularly evaluate and evolve the architecture as usage grows

- ▶ Architectures must evolve to keep pace with growth and change
- ▶ What works today may fall short tomorrow
- ▶ Static designs often turn into bottlenecks over time
- ▶ Continuous improvements drive scalability and long-term success
- ▶ Examples:
 - ▶ Migrating from monolithic applications to microservices for flexibility
 - ▶ Using containerization for better portability between clouds
 - ▶ Replacing custom-built components with AWS managed services as needs expand



Bonus Tip: Build vs. Buy: Making the Right Choice

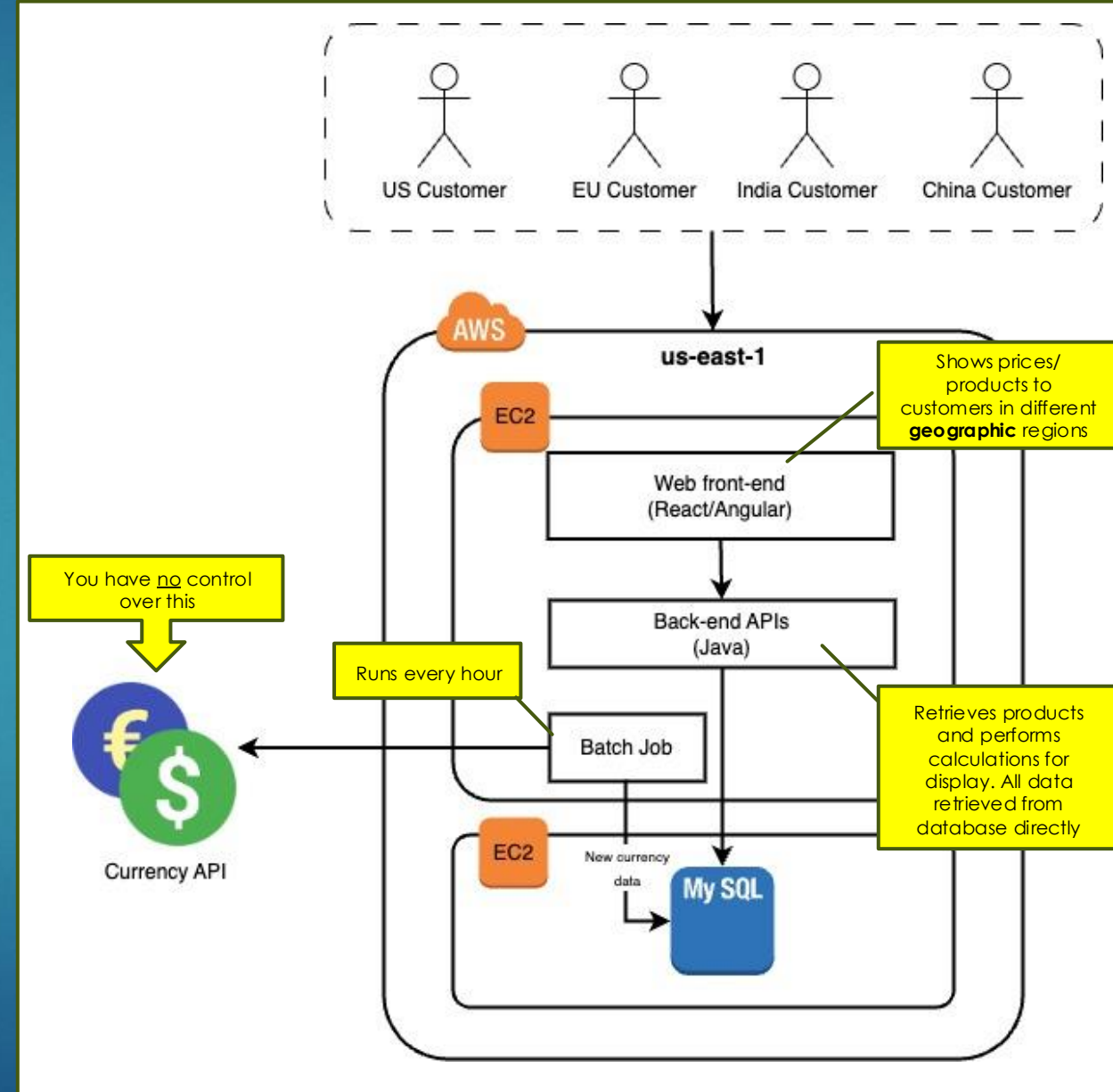
- ▶ Development teams often struggle to accept that buying can sometimes be better than building
- ▶ For commodity solutions, evaluate SaaS options and compare costs objectively
- ▶ While “buy” may involve higher upfront costs (e.g., licensing), it often reduces long-term support and maintenance burdens
- ▶ Avoid “Not Invented Here Syndrome” (NIHS)
 - ▶ NIHS is the tendency to reject external solutions simply because they weren’t developed internally
 - ▶ This mindset can lead to unnecessary complexity and missed opportunities for efficiency



DISCUSSION TOPICS

21

- ▶ Review the provided architecture through the lens of best practices we covered
- ▶ Your goal: recommend improvements for scalability, resilience, and simplicity
- ▶ Focus on:
 - ▶ **Risk Areas:** Where potential failures or bottlenecks exist
 - ▶ **Redesign Opportunities:** Components that need rethinking for better performance
 - ▶ **Managed Services:** Identify AWS services that can replace custom solutions and reduce complexity
- ▶ Instructions:
 - ▶ Each team has 10 minutes to analyze and propose a better architecture
 - ▶ Be ready to share specific recommendations and justify your choices
- ▶ Template and diagram can be found here:
 - ▶ [Assignments > Activity - AWS Architecture Recommendations](#)



Example Solution

- ▶ CloudFront + S3 for global content delivery
- ▶ Managed database (RDS or DynamoDB)
- ▶ Serverless compute for low maintenance
- ▶ Monitoring and alerting built-in from day one

